Proiectarea Algoritmilor

Curs 2 – Scheme de algoritmi – Greedy



Bibliografie

- Cormen – Introducere în Algoritmi cap. Algoritmi Greedy (17)
- Giumale – Introducere în Analiza Algoritmilor cap 4.4
- https://people.cs.umass.edu/~barring/cs611/lecture/4.pdf
- http://profs.info.uaic.ro/~dlucanu/cursuri/tpaa/resurse/Curs6.pps
- http://euler.math.fau.edu/locke/Greedy.htm
- http://en.wikipedia.org/wiki/Greedoid
- https://colab.research.google.com/gist/chirucos/5d89e4e6f3129821afeedbb280d5a358

Greedy (I)

- Metodă de rezolvare eficientă a unor probleme de optimizare.
- Soluția trebuie să satisfacă un criteriu de optim global (greu de verificat) → optim local mai ușor de verificat.
- Se aleg soluții parțiale ce sunt îmbunătățite repetat pe baza criteriului de optim local până ce se obțin soluții finale.
- Soluțiile parțiale ce nu pot fi îmbunătățite sunt abandonate → proces de rezolvare irevocabil (fără reveniri)!

Greedy (II)

- Schema generală de rezolvare a unei probleme folosind Greedy (programarea lacomă):

- Rezolvare_lacomă(Crit_optim, Problemă)
  - 1. sol_parțiale = sol_inițiale(Problemă); // determinarea soluțiilor parțiale
  - 2. sol_fin = Φ;
  - 3. **Cât timp** (sol_parțiale ≠ Φ)
  - 4. **Pentru fiecare** (s **în** sol_parțiale)
  - 5. **Dacă** (s este o soluție a problemei) { // dacă e soluție
  - 6. sol_fin = sol_fin U {s}; // finală se salvează
  - 7. sol_parțiale = sol_parțiale \ {s};
  - 8. } **Altfel** // se poate optimiza?
  - 9. **Dacă** (optimizare_posibilă (s, Crit_optim, Problemă))
  - 10. sol_parțiale = sol_parțiale \ {s} U // da optimizare(s, Crit_optim, Problemă)
  - 11. **Altfel** sol_parțiale = sol_parțiale \ {s}; // nu
  - 12. **Întoarce** sol_fin;

Comparație D&I și Greedy

- Tip abordare
  - D&I: top-down;
  - Greedy: top-down.
- Criteriu de optim
  - D&I: nu;
  - Greedy: da.

Arbori Huffman

- Metodă de codificare folosită la compresia fișierelor.
- Construcția unui astfel de arbore se realizează printr-un algoritm Greedy.
- Considerăm un text, de exemplu:
  - **ana are mere**
- Vom exemplifica pas cu pas construcția arborelui de codificare pentru acest text și vom defini pe parcurs conceptele de care avem nevoie.



Arbori Huffman – Definiții (I)

- K – mulțimea de simboluri ce vor fi codificate. (a, n, “ “, r, e, m)
- Arbore de codificare a cheilor K este un arbore binar ordonat (păstrează ordinea copiilor fiecărui nod) cu proprietățile:
  - Doar frunzele conțin cheile din K; nu există mai mult de o cheie într-o frunză;
  - Toate nodurile interne au exact 2 copii;
  - Arcele sunt codificate cu 0 și 1 (arcul din stânga unui nod – codificat cu 0).
- k = Codul unei chei – este șirul etichetelor de pe calea de la rădăcina arborelui la frunza care conține cheia k (k este din K).
- p(k) – frecvența de apariție a cheii k în textul ce trebuie comprimat.
- Ex pentru “ana are mere”:
  - p(a) = p(e) = 0.25; p(n) = p(m) = 0.083; p(r) = p( ) = 0.166

Arbori Huffman – Definiții (II)

- A – arborele de codificare a cheilor.
- lg_cod(k) – lungimea codului cheii k conform A.
- nivel(k,A) – nivelul pe care apare în A frunza ce conține cheia K.
- Costul unui arbore de codificare A al unor chei K relativ la o frecventa p este:

$$Cost\,(A) = \sum_{k \in K} lg_cod(k) * p(k) = \sum_{k \in K} nivel\,(k, A) * p(k)$$

- Un arbore de codificare cu cost minim al unor chei K, relativ la o frecvență p este un arbore Huffman, iar codurile cheilor sunt coduri Huffman.

Arbori Huffman – algoritm de construcție (I)

- 1. Pentru fiecare k din K se construiește un arbore cu un singur nod care conține cheia k si este caracterizat de ponderea w = p(k). Subarborii construiți formează o mulțime numită Arb.

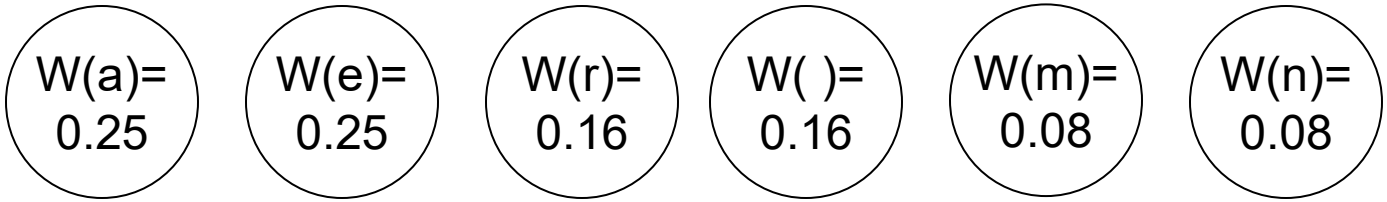
- 2. Se aleg doi subarbori a şi b din Arb astfel încât a şi b au pondere minimă.

Arbori Huffman – algoritm de construcție (II)

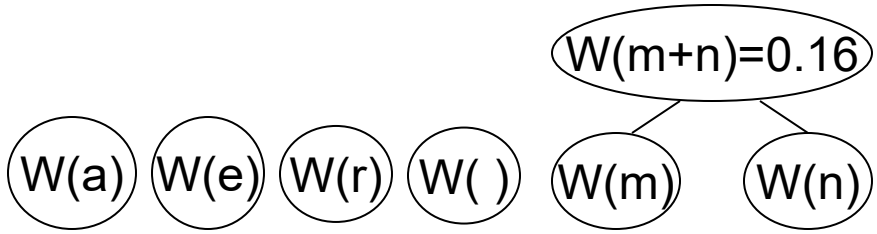
- 3. Se construiește un arbore binar cu o rădăcina *r* care nu conține nici o cheie și cu descendenții a și b. Ponderea arborelui este definită ca w(r) = w(a) + w(b).
- 4. Arborii a și b sunt eliminați din Arb iar r este inserat în Arb.
- 5. Se repetă procesul de construcție descris de pașii 2-4 până când mulțimea Arb conține un singur arbore – Arborele Huffman pentru cheile K.

Arbori Huffman – Exemplu (I)

- Text: ana are mere
- p(a) = p(e) = 0.25; p(n) = p(m) = 0.083; p(r) = p( ) = 0.166
- Pasul 1:

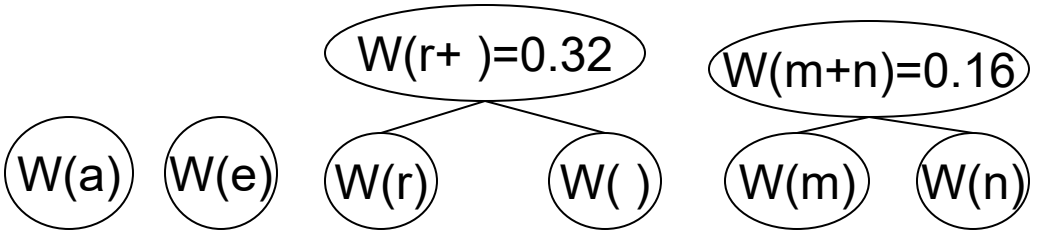


- Pasii 2-4:

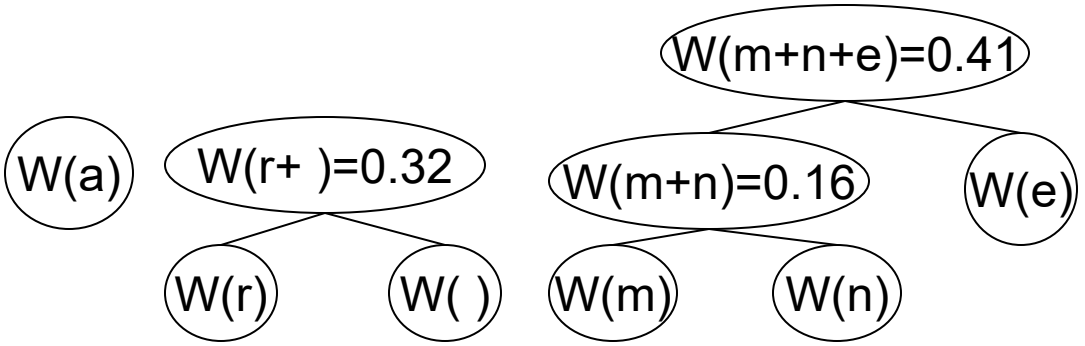


Arbori Huffman – Exemplu (II)

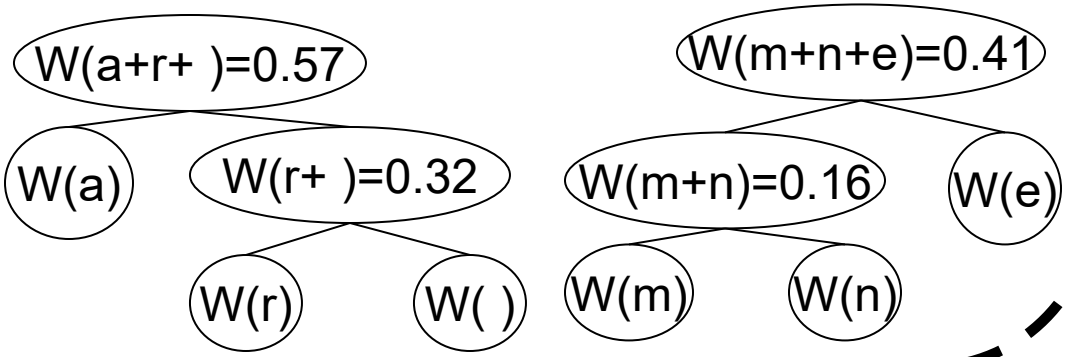
- Pasii 2-4 (II):



- Pasii 2-4 (III):

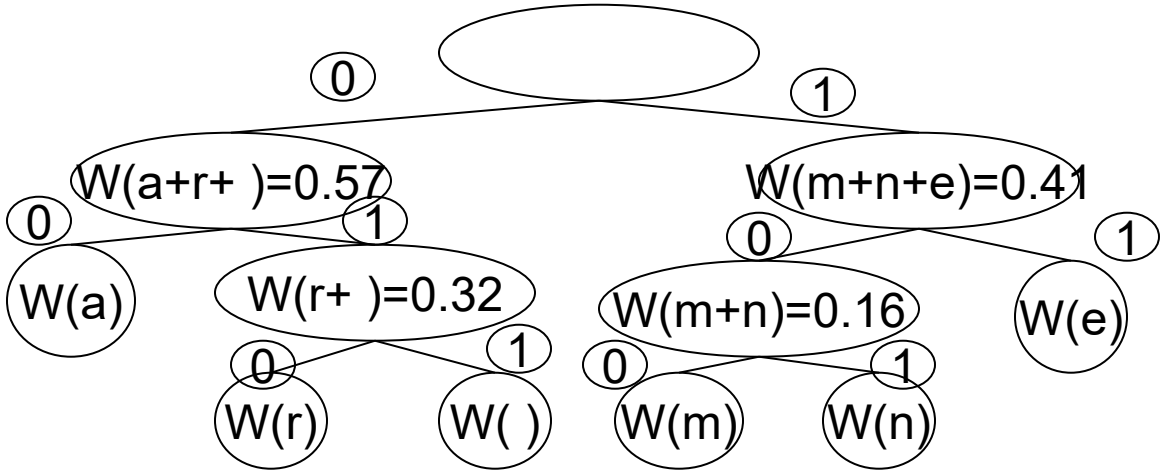


- Pasii 2-4 (IV):



Arbori Huffman – Exemplu (III)

- Pasii 2-4 (V):



- Codificare: a - 00; e -11; r - 010; ’ ‘ - 011; m - 100; n - 101;

$$Cost(A) = \sum_{k \in K} lg_cod(k) * p(k) = \sum_{k \in K} nivel(k, A) * p(k)$$

- p(a) = p(e) = 0.25; p(n) = p(m) = 0.083; p(r) = p( ) = 0.166
- Cost(A) = 2 * 0.25 + 2 * 0.25 + 3 * 0.083 + 3 * 0.083 + 3 * 0.166 + 3 * 0.166 = 1 + 1.5 = 2.5 biti.

Arbori Huffman - pseudocod

- Huffman(K,p){
  - 1. Arb = {frunză(k, p(k)) | k ∈ K};
  - 2. **Cât timp** (card (Arb) > 1) // am mai mulți subarbori
  - 3. Fie a_1 și a_2 arbori din Arb a.i. ∀a ∈ Arb a ≠ a_1 si a ≠ a_2, avem $w(a_1) \leq w(a)$ și $w(a_2) \leq w(a)$); // practic se extrage // de două ori minimul și se salvează în a_1 și a_2
  - 4. Arb = Arb \ {a_1, a_2} U nod_intern(a_1, a_2, $w(a_1) + w(a_2)$);
  - 5. **Dacă** (Arb = Φ)
  - 6. **Întoarce** arb_vid;
  - 6. **Altfel**
  - 7. fie A singurul arbore din mulțimea Arb;
  - 8. **Întoarce** A;

- Notații folosite:
  - a = frunză (k, p(k)) – subarbore cu un singur nod care conține cheia k, iar w(a) = p(k);
  - a = nod_intern(a_1, a_2, x) – subarbore format dintr-un nod intern cu descendenții a_1 și a_2 și w(a) = x.

Arbori Huffman - Decodificare

- Se încarcă arborele și se decodifică textul din fișier conform algoritmului:

- Decodificare (in, out)

```
    A = restaurare_arbore (in)        // reconstruiesc arborele
  Cât timp (! terminare_cod(in)) // mai am caractere de citit
      nod = A                          // pornesc din rădăcină
      Cât timp (! frunză(nod))     // cât timp nu am determinat caracterul
          Dacă (bit(in) = 1) nod = dreapta(nod)      // avansez în arbore
              Altfel nod = stânga(nod)
      Scrie (out, cheie(nod))       // am determinat caracterul și îl scriu la
                                                  // ieșire
```

Demonstrație (I)

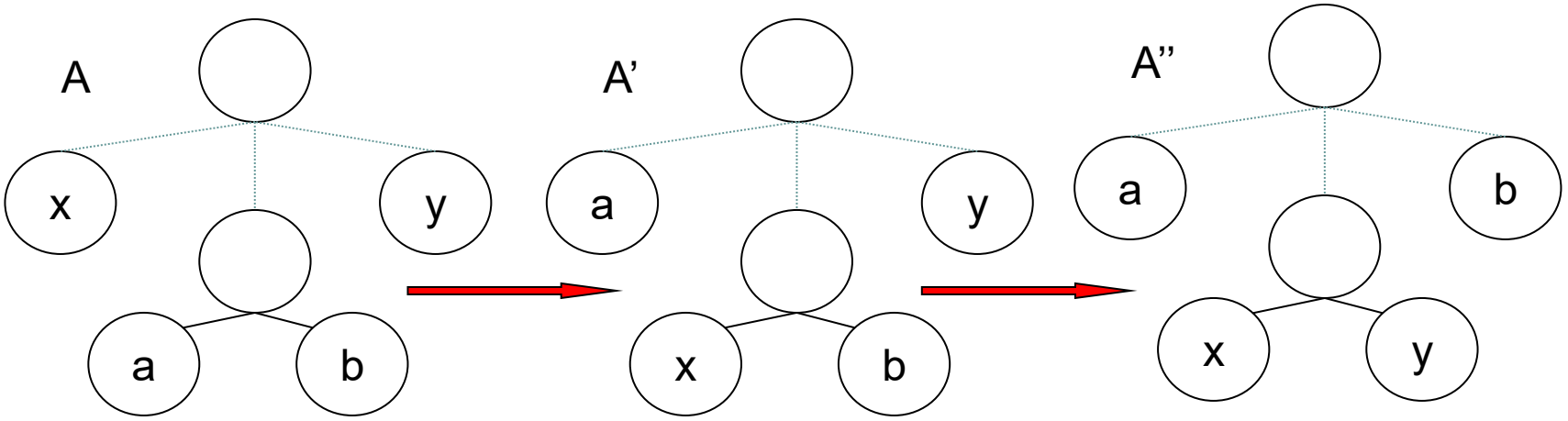
- Arborele de codificare construit trebuie să aibă cost minim pentru a fi arbore Huffman.

- Lema 1. Fie K mulțimea cheilor dintr-un arbore de codificare, card(K) ≥ 2, x, y două chei cu pondere minimă. ∃ un arbore Huffman de înălțime h în care cheile x şi y apar pe nivelul h fiind descendente ale aceluiași nod intern.

Demonstrație (II)

- Demonstrație Lema 1:

$$Cost(A) = \sum_{k \in K} lg_cod(k) * p(k) = \sum_{k \in K} nivel(k, A) * p(k)$$



- Se interschimbă a cu x şi b cu y şi din definiţia costului arborelui => cost(A'') ≤ cost(A') ≤ cost(A) => A'' arbore Huffman

Demonstrație (III)

- Lema 2. Fie A un arbore Huffman cu cheile K, iar x şi y două chei direct descendente ale aceluiaşi nod intern *a*. Fie K' = K \ {x,y} U {z} unde z este o cheie fictivă cu ponderea w(z) = w(x) + w(y). Atunci arborele A' rezultat din A prin înlocuirea subarborelui cu rădăcina *a* și frunzele x, y cu subarborele cu un singur nod care conţine frunza z, este un arbore Huffman cu cheile K'.

- Demonstrație:
  - 1) analog Cost(A') ≤ Cost(A) (Cost(A) = Cost(A') + w(x) + w(y))
  - 2) pp există A'' a.i. Cost(A'') < Cost(A') =>
    - Cost(A'') < Cost(A) - (w(x) + w(y));
    - Cost(A'') + w(x) + w(y) < Cost(A); => A nu este Huffman (contradicţie)

Demonstrație (IV)

- Teoremă – Algoritmul Huffman construiește un arbore Huffman.
- Demonstrație: prin inducție după numărul de chei din mulțimea K.
- $n \leq 2$ => evident
- $n > 2$
  - Ip. Inductivă: algoritmul Huffman construiește arbori Huffman pentru orice mulțime cu n-1 chei
  - Fie $K = \{k_1, k_2, \dots, k_n\}$ a.i. $w(k_1) \leq w(k_2) \leq \dots \leq w(k_n)$

Demonstrație (V)

- Cf. Lema 1, ∃ Un arbore Huffman unde cheile k_1, k_2 sunt pe același nivel şi descendente ale aceluiași nod.
- A_{n-1} – un arbore cu n-1 chei K' = K - $\{k_1,k_2\} \cup z$ unde w(z) = $w(k_1) + w(k_2)$.
- A_{n-1} rezultă din A_n prin modificările prezentate in Lema 2 => A_{n-1} este Huffman, şi cf. ipotezei inductive e construit prin algoritmul Huffman(K',p').
- => Algoritmul Huffman(K, p) construiește arborele format din k_1 si k_2 si apoi lucrează ca algoritmul Huffman(K', p') ce construiește A_{n-1} => construiește arborele Huffman(K, p).

Alt exemplu (I)

Problema rucsacului

Trebuie să umplem un rucsac de capacitate maximă M kg cu obiecte care au greutatea m_i şi valoarea v_i. Putem alege mai multe obiecte din fiecare tip cu scopul de a maximiza valoarea obiectelor din rucsac.

- Varianta 1: putem alege fracţiuni de obiect – "problema continuă"
- Varianta 2: nu putem alege decât obiecte întregi (număr natural de obiecte din fiecare tip) – "problema 0-1"



Alt exemplu (II)

- Varianta 1: Algoritm Greedy
  - sortăm obiectele după raportul v_i/m_i;
  - adăugăm fracţiuni din obiectul cu cea mai mare valoare per kg până epuizăm stocul şi apoi adăugăm fracţiuni din obiectul cu valoarea următoare.
  - Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10
  - Soluție: (m_2 , v_2) 8kg şi 2kg din (m_1,v_1) sau din (m_4,v_4) – valoarea totală: 19 + 2 * 10 / 5 = 23
- Varianta 2: Algoritmul Greedy nu funcţionează => contraexemplu
  - Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10
  - Rezultat corect – 2 obiecte: (m_1,v_1) și (m_4,v_4) – valoarea totală: 20
  - Rezultat algoritm Greedy – 1 obiect (m_2,v_2) – valoarea totală: 19

Când funcţionează algoritmii Greedy? (I)

- Problema are proprietatea de substructură optimă
  - Soluţia problemei conţine soluţiile subproblemelor.
- Problema are proprietatea alegerii locale
  - Alegând soluţia optimă local se ajunge la soluţia optimă global.



Când funcţionează algoritmii Greedy? (II)

- Fie E o mulţime finită nevidă şi I $\subset \mathcal{P}$(E) a.i. $\varnothing \in$ I, $\forall$X $\subseteq$ Y şi Y $\in$ I => X $\in$ I. Atunci spunem că (E,I) este un sistem accesibil.

- Submulţimile din I sunt numite submulţimi "independente".

- Exemple:
  - Ex1: E = {e_1, e_2, e_3} si I = {$\varnothing$, {e_1}, {e_2}, {e_3}, {e_1, e_2}, {e_2, e_3}} – mulțimile ce nu conțin e_1 si e_3.
  - Ex2: E – muchiile unui graf neorientat şi I mulţimea mulţimilor de muchii ce nu conţin un ciclu (mulțimea arborilor).
  - Ex3: E set de vectori dintr-un spaţiu vectorial, I mulţimea mulţimilor de vectori linear independenţi.
  - Ex4: E – muchiile unui graf neorientat şi I mulţimea mulţimilor de muchii în care oricare 2 muchii nu au un vârf comun.

Când funcţionează algoritmii Greedy? (III)

- Un sistem accesibil este un matroid dacă satisface proprietatea de interschimbare:

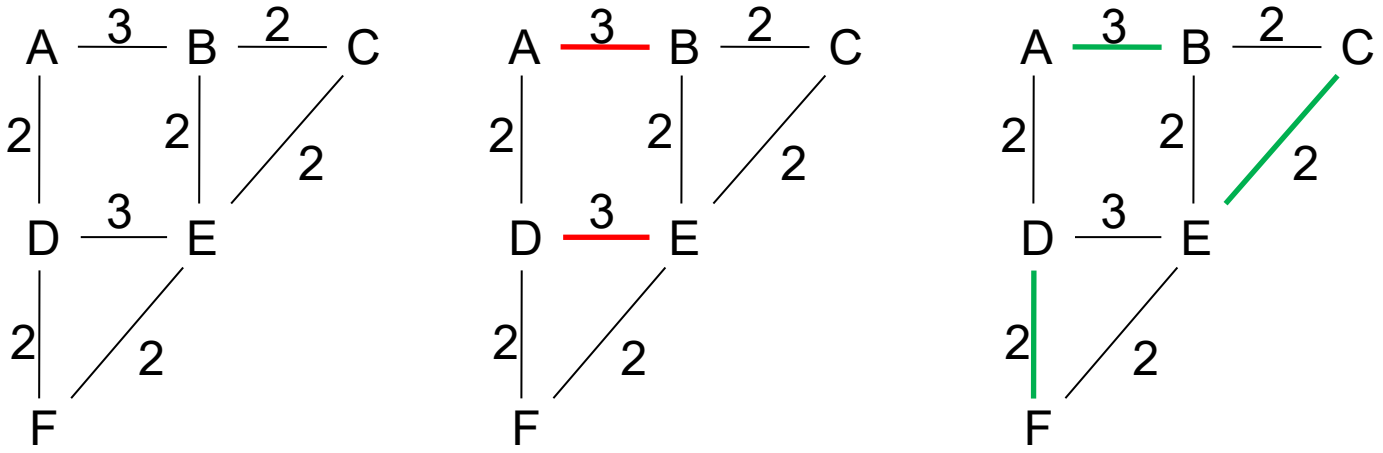
  $\forall X, Y \in I$ şi $|X| < |Y| \Rightarrow \exists e \in Y \setminus X$ a. î. $X \cup \{e\} \in I$

- Teoremă. Dacă (E, I) este matroid atunci există un algoritm Greedy care rezolvă problema de optimizare.

Verificăm exemplele

- Ex1: I = {∅, {e_1}, {e_2}, {e_3}, {e_1, e_2}, {e_2, e_3}}
- De exemplu, fie Y = {e_1, e_2} și X = {e_1}
- → Y \ X = {e_2} → X ∪ {e_2} ∈ I → dacă proprietatea e validă pentru toate perechile de mulțimi X, Y → matroid

- Ex4:



Problema rucsacului discretă (varianta 2)

- Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10

- Fie I = mulțimea subsoluțiilor și X = $\{m_2\}$ și Y = $\{m_1, m_4\}$ → $m_1 \in Y \setminus X$ dar $X \cup \{m_1\} \notin I$ → problema nu respectă proprietatea de matroid → problema nu se poate rezolva folosind tehnica Greedy!

Greedy – tema de gândire

- Se dă un număr natural n. Să se găsească cel mai mare subset S din {1, 2, ..., n} astfel încât nici un element din S să nu fie divizibil cu nici un alt element din S.
- În plus, dacă există mai multe subseturi maximale ce respectă proprietatea de mai sus, să se determine întotdeauna cel mai mic din punct de vedere lexicografic.
- S este lexicografic mai mic decât T dacă cel mai mic element care este membru din S sau T, dar nu din amândouă, face parte din S.

ÎNTREBĂRI?

